06. L2OutputOracle 시스템 분석

개요

L2OutputOracle 시스템은 OPinit에서 L2 블록의 상태를 L1에 제출하고 검증하는 핵심 메커니즘입니다. 이 시스템은 Proposer가 L2 블록의 output root를 L1에 제출하고, Challenger가 잘못된 제출을 모니터링하여 삭제할 수 있는 구조로 설계되었습니다.

핵심 컴포넌트

1. Output 데이터 구조

```
// Output is a l2 block submitted by proposer.
type Output struct {
// Hash of the l2 output.
OutputRoot []byte  `protobuf:"bytes,1,opt,name=output_root,json=outputRoot,proto
// The l1 block number that the output root was submitted in.
L1BlockNumber  uint64  `protobuf:"varint,2,opt,name=l1_block_number,json=l1Block
// Timestamp of the l1 block that the output root was submitted in.
L1BlockTime time.Time `protobuf:"bytes,3,opt,name=l1_block_time,json=l1BlockTime
// The l2 block number that the output root was submitted in.
L2BlockNumber  uint64  `protobuf:"varint,4,opt,name=l2_block_number,json=l2Block
}
```

2. BridgeConfig의 Finalization 설정

```
type BridgeConfig struct {
// The address of the challenger.
Challenger  string
// The address of the proposer.
Proposer  string
// The minimum time duration that must elapse before a withdrawal can be finaliz
FinalizationPeriod time.Duration
// Other fields...
}
```

MsgProposeOutput 함수 동작 분석

1. 메시지 구조

```
message MsgProposeOutput {
string proposer = 1;
uint64 bridge_id = 2;
uint64 output_index = 3;
uint64 l2_block_number = 4;
bytes output_root = 5;
```

```
}
```

2. ProposeOutput 함수 구현

```
func (ms MsgServer) ProposeOutput(ctx context.Context, req *types.MsgProposeOutp

sdkCtx := sdk.UnwrapSDKContext(ctx)

// 1. 메시지 검증

if err := req.Validate(ms.authKeeper.AddressCodec()); err != nil {

return nil, err

}


// 2. 브리지 설정 조회 및 권한 확인

bridgeConfig, err := ms.GetBridgeConfig(ctx, req.BridgeId)

if err != nil {

return nil, err

}


// 3. Proposer 권한 확인

if bridgeConfig.Proposer != req.Proposer {

return nil, types.ErrUnauthorized

}


// 4. Output 인덱스 검증
```

```
nextOutputIndex, err := ms.GetNextOutputIndex(ctx, req.BridgeId)
if err != nil {
return nil, err
}

if req.OutputIndex != nextOutputIndex {
return nil, types.ErrInvalidOutputIndex
}

// 5. Output 제안 저장
output := types.Output{
OutputRoot: req.OutputRoot,
L1BlockNumber: uint64(sdkCtx.BlockHeight()),
L1BlockTime: sdkCtx.BlockTime(),
L2BlockNumber: req.L2BlockNumber,
}

if err := ms.SetOutputProposal(ctx, req.BridgeId, req.OutputIndex, output); er
return nil, err
}

// 6. 다음 Output 인덱스 증가
if _, err := ms.IncreaseNextOutputIndex(ctx, req.BridgeId); err != nil {
```

```
return  nil, err
}

// 7. 이벤트 발생
sdkCtx.EventManager().EmitEvent(sdk.NewEvent(
types.EventTypeProposeOutput,
sdk.NewAttribute(types.AttributeKeyProposer, req.Proposer),
sdk.NewAttribute(types.AttributeKeyBridgeId, strconv.FormatUint(req.BridgeId, 10
sdk.NewAttribute(types.AttributeKeyOutputIndex, strconv.FormatUint(req.OutputInd
sdk.NewAttribute(types.AttributeKeyOutputRoot, hex.EncodeToString(req.OutputRoot
sdk.NewAttribute(types.AttributeKeyL2BlockNumber, strconv.FormatUint(req.L2Block
))

return &types.MsgProposeOutputResponse{}, nil
}
```

3. 메시지 검증 로직

```
func (msg MsgProposeOutput) Validate(ac address.Codec) error {
// Proposer 주소 검증
_, err := ac.StringToBytes(msg.Proposer)
if  err != nil {
return  err
```

```
}

// Bridge ID 검증
if msg.BridgeId == 0 {
return ErrInvalidBridgeId
}

// Output Root 길이 검증 (32바이트 해시)
if len(msg.OutputRoot) != 32 {
return ErrInvalidHashLength.Wrap("output_root")
}

return nil
}
```

MsgDeleteOutput 함수 동작 분석

1. 메시지 구조

```
message MsgDeleteOutput {
string challenger = 1;
uint64 bridge_id = 2;
uint64 output_index = 3;
```

```
}
```

2. DeleteOutput 함수 구현

```
func (ms MsgServer) DeleteOutput(ctx context.Context, req *types.MsgDeleteOutput

// 1. 메시지 검증

if err := req.Validate(ms.authKeeper.AddressCodec()); err != nil {

return nil, err

}


// 2. 브리지 설정 조회

bridgeConfig, err := ms.GetBridgeConfig(ctx, req.BridgeId)

if err != nil {

return nil, err

}


// 3. 권한 확인 (Challenger, Proposer, 또는 거버넌스)

if bridgeConfig.Challenger != req.Challenger &&

bridgeConfig.Proposer != req.Challenger &&

ms.GetAuthority() != req.Challenger {

return nil, types.ErrUnauthorized

}
```

```go
// 4. 다음 Output 인덱스 조회
nextOutputIndex, err := ms.GetNextOutputIndex(ctx, req.BridgeId)
if err != nil {
return nil, err
}

// 5. 삭제할 Output 인덱스 범위 검증
if req.OutputIndex >= nextOutputIndex {
return nil, types.ErrInvalidOutputIndex
}

// 6. 지정된 인덱스부터 마지막까지 모든 Output 삭제
for i := req.OutputIndex; i < nextOutputIndex; i++ {
if err := ms.DeleteOutputProposal(ctx, req.BridgeId, i); err != nil {
return nil, err
}
}

// 7. 다음 Output 인덱스 재설정
if err := ms.SetNextOutputIndex(ctx, req.BridgeId, req.OutputIndex); err != ni
return nil, err
}
```

```
// 8. 이벤트 발생
sdk.UnwrapSDKContext(ctx).EventManager().EmitEvent(sdk.NewEvent(
types.EventTypeDeleteOutput,
sdk.NewAttribute(types.AttributeKeyChallenger, req.Challenger),
sdk.NewAttribute(types.AttributeKeyBridgeId, strconv.FormatUint(req.BridgeId, 10
sdk.NewAttribute(types.AttributeKeyOutputIndex, strconv.FormatUint(req.OutputInd
))

return &types.MsgDeleteOutputResponse{}, nil
}
```

3. DeleteOutputProposal 핵심 로직

```
func (k Keeper) DeleteOutputProposal(ctx context.Context, bridgeId, outputIndex
// 1. Output 조회
output, err := k.GetOutputProposal(ctx, bridgeId, outputIndex)
if err != nil {
return err
}

// 2. Finalization 상태 확인
if isFinalized, err := k.isFinalized(ctx, bridgeId, output); err != nil {
```

```
return err
} else if isFinalized {
return types.ErrAlreadyFinalized
}


// 3. Output 삭제
return k.OutputProposals.Remove(ctx, collections.Join(bridgeId, outputIndex))
}
```

Output Root 생성 및 검증 프로세스

1. Output Root 생성 과정

L2에서 Output Root는 다음과 같은 과정을 통해 생성됩니다:

1. **L2 블록 실행**: L2에서 트랜잭션들이 실행되어 새로운 상태가 생성됩니다
2. **상태 루트 계산**: 새로운 L2 상태에서 Merkle Tree 루트를 계산합니다
3. **Output Root 구성**: 상태 루트와 기타 메타데이터를 조합하여 Output Root를 생성합니다
4. **L1 제출**: Proposer가 `MsgProposeOutput` 을 통해 L1에 제출합니다

2. 검증 프로세스

```
// Output 제안 시 검증 항목
func validateOutputProposal(output types.Output, bridgeConfig types.BridgeConfi
// 1. Output Root 길이 검증 (32바이트)
```

```
if len(output.OutputRoot) != 32 {
return ErrInvalidHashLength
}


// 2. L2 블록 번호 순서 검증
// (이전 Output보다 큰 L2 블록 번호여야 함)
// 3. 시간 순서 검증
// (L1 블록 시간이 이전 Output보다 나중이어야 함)
return nil
}
```

3. Finalization 검증

```
func (k Keeper) isFinalizedWithConfig(ctx context.Context, bridgeConfig types.Br
currentTime := sdk.UnwrapSDKContext(ctx).BlockTime()
finalizationTime := output.L1BlockTime.Add(bridgeConfig.FinalizationPeriod)
return currentTime.Unix() >= finalizationTime.Unix(), nil
}
```

Challenger의 모니터링 및 챌린지 메커니즘

1. Challenger의 역할

Challenger는 다음과 같은 역할을 수행합니다:

1. **지속적 모니터링**: L1에 제출된 모든 Output 제안을 모니터링
2. **유효성 검증**: 자체 L2 노드를 실행하여 Output Root의 정확성 검증
3. **챌린지 실행**: 잘못된 Output 발견 시 `MsgDeleteOutput` 으로 삭제

2. 모니터링 프로세스

```go
// Challenger의 모니터링 로직 (의사코드)
func (c *Challenger) MonitorOutputs() {
for {
// 1. 새로운 Output 제안 감지
newOutputs := c.getNewOutputProposals()
for _, output := range newOutputs {
// 2. 자체 L2 노드에서 동일한 블록 실행
expectedRoot := c.l2Node.ExecuteBlock(output.L2BlockNumber)
// 3. Output Root 비교
if !bytes.Equal(expectedRoot, output.OutputRoot) {
// 4. 잘못된 Output 발견 시 챌린지
c.challengeOutput(output.BridgeId, output.OutputIndex)
}
}
time.Sleep(c.monitoringInterval)
}
}
```

3. 챌린지 권한 체계

```
// 삭제 권한 확인 로직
func canDeleteOutput(challenger string, bridgeConfig types.BridgeConfig, autho
return challenger == bridgeConfig.Challenger || // 지정된 Challenger
challenger == bridgeConfig.Proposer || // Proposer 자체 삭제
challenger == authority // 거버넌스 권한
}
```

4. 챌린지 시나리오

시나리오 1: 정상적인 챌린지

```
// 1. Challenger가 잘못된 Output 발견
// 2. MsgDeleteOutput 제출
msg := types.NewMsgDeleteOutput(challengerAddr, bridgeId, outputIndex)

// 3. 권한 확인 후 삭제 실행
// 4. 해당 인덱스부터 모든 후속 Output 삭제
```

시나리오 2: Proposer의 자체 수정

```
// 1. Proposer가 자신의 실수 발견
// 2. 자체적으로 잘못된 Output 삭제
```

```
msg := types.NewMsgDeleteOutput(proposerAddr, bridgeId, outputIndex)


// 3. 올바른 Output 재제출
correctMsg := types.NewMsgProposeOutput(proposerAddr, bridgeId, outputIndex, cor
```

블록 Finalization과 Withdrawal Proving 연동

1. Finalization 메커니즘

```
func (k Keeper) GetLastFinalizedOutput(ctx context.Context, bridgeId uint64) (o
bridgeConfig, err := k.GetBridgeConfig(ctx, bridgeId)
if err != nil {
return outputIndex, outputProposal, err
}


// 역순으로 Output들을 순회하며 Finalized된 것 찾기
cb := func(key collections.Pair[uint64, uint64], output types.Output) (stop boo
if ok, err := k.isFinalizedWithConfig(ctx, bridgeConfig, output); err != nil {
return true, err
} else if ok {
outputIndex = key.K2()
outputProposal = output
return true, nil // 첫 번째 Finalized Output 발견 시 중단
```

```
}
return false, nil
}
if err := k.ReverseIterateOutputProposals(ctx, bridgeId, cb); err != nil {
return outputIndex, outputProposal, err
}


return outputIndex, outputProposal, nil
}
```

2. Withdrawal Proving 연동

Withdrawal proving 과정에서 L2OutputOracle의 역할:

1. **Finalized Output 확인**: 출금하려는 트랜잭션이 포함된 L2 블록이 finalized되었는지 확인
2. **Merkle Proof 검증**: 제출된 Merkle proof가 finalized된 Output Root와 일치하는지 검증
3. **시간 제약 확인**: Finalization period가 경과했는지 확인

```
// Withdrawal proving에서 사용되는 검증 로직
func (k Keeper) VerifyWithdrawalProof(ctx context.Context, bridgeId uint64, l2B
// 1. 해당 L2 블록이 포함된 Output 찾기
output, err := k.findOutputByL2Block(ctx, bridgeId, l2BlockNumber)
if err != nil {
return err
}
```

```go
// 2. Output이 finalized되었는지 확인
isFinalized, err := k.isFinalized(ctx, bridgeId, output)
if err != nil {
return err
}
if !isFinalized {
return types.ErrNotFinalized
}

// 3. Merkle proof 검증
if !verifyMerkleProof(proof, output.OutputRoot) {
return types.ErrInvalidProof
}

return nil
}
```

3. Finalization Period 관리

```go
// Finalization Period 업데이트 (거버넌스를 통해서만 가능)
func (ms MsgServer) UpdateFinalizationPeriod(ctx context.Context, req *types.Msg
```

```go
// 1. 권한 확인 (거버넌스만 가능)
if req.Authority != ms.GetAuthority() {
return nil, types.ErrUnauthorized
}

// 2. 유효한 기간인지 확인
if req.FinalizationPeriod <= 0 {
return nil, types.ErrInvalidFinalizationPeriod
}

// 3. 브리지 설정 업데이트
bridgeConfig, err := ms.GetBridgeConfig(ctx, req.BridgeId)
if err != nil {
return nil, err
}

bridgeConfig.FinalizationPeriod = req.FinalizationPeriod
if err := ms.SetBridgeConfig(ctx, req.BridgeId, bridgeConfig); err != nil {
return nil, err
}

return &types.MsgUpdateFinalizationPeriodResponse{}, nil
```

```
}
```

보안 고려사항

1. 시간 기반 보안

- **Finalization Period**: 충분한 시간을 두어 Challenger가 잘못된 Output을 발견할 수 있도록 함
- **Challenge Window**: Output 제출 후 일정 시간 동안만 챌린지 가능

2. 권한 기반 보안

- **Proposer 권한**: 오직 지정된 Proposer만 Output 제출 가능
- **Challenger 권한**: 지정된 Challenger, Proposer 본인, 또는 거버넌스만 삭제 가능
- **거버넌스 권한**: 최종적인 분쟁 해결 및 시스템 파라미터 변경

3. 데이터 무결성

- **Output Root 검증**: 32바이트 해시 길이 강제
- **순서 보장**: Output 인덱스의 순차적 증가 보장
- **Finalization 보호**: Finalized된 Output은 삭제 불가

이벤트 및 모니터링

1. ProposeOutput 이벤트

```
sdk.NewEvent(
types.EventTypeProposeOutput,
```

```
sdk.NewAttribute(types.AttributeKeyProposer, proposer),
sdk.NewAttribute(types.AttributeKeyBridgeId, bridgeId),
sdk.NewAttribute(types.AttributeKeyOutputIndex, outputIndex),
sdk.NewAttribute(types.AttributeKeyOutputRoot, outputRoot),
sdk.NewAttribute(types.AttributeKeyL2BlockNumber, l2BlockNumber),
)
```

2. DeleteOutput 이벤트

```
sdk.NewEvent(
types.EventTypeDeleteOutput,
sdk.NewAttribute(types.AttributeKeyChallenger, challenger),
sdk.NewAttribute(types.AttributeKeyBridgeId, bridgeId),
sdk.NewAttribute(types.AttributeKeyOutputIndex, outputIndex),
)
```

이러한 이벤트들을 통해 외부 시스템들이 L2OutputOracle의 상태 변화를 실시간으로 모니터링할 수 있습니다.

결론

L2OutputOracle 시스템은 OPinit의 핵심 보안 메커니즘으로, Proposer와 Challenger 간의 균형을 통해 L2 상태의 무결성을 보장합니다. Finalization period와 챌린지 메커니즘을 통해 잘못된 상태 제출을 방지하고, withdrawal proving과의 연동을 통해 안전한 자산 이동을 가능하게 합니다.